

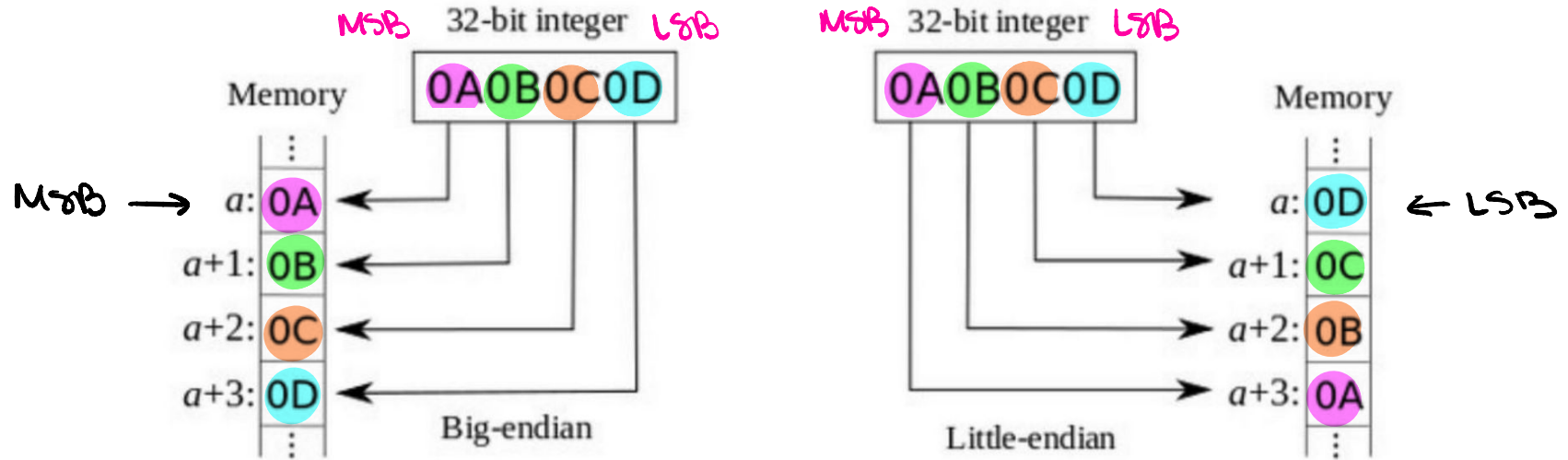
Recitation 13: Endianism, Stack, and Syscalls

CS-UY 2214

Endianism

- Every memory address can store 1 byte
 - So, a 32-bit value takes 4 bytes to store
- Endianism: The order in which multibyte values are loaded into memory
 - Little Endian: LSB is stored at starting address
 - Big Endian: MSB is stored at starting address
- x86 uses Little Endian architecture!!!

Little vs Big Endian

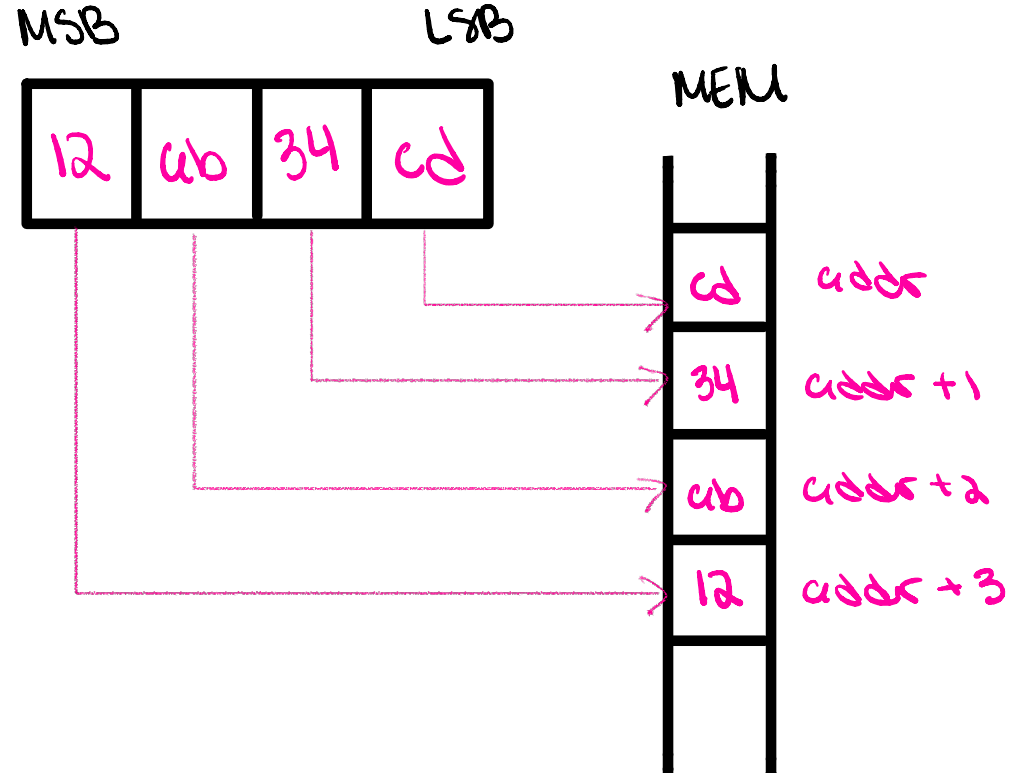
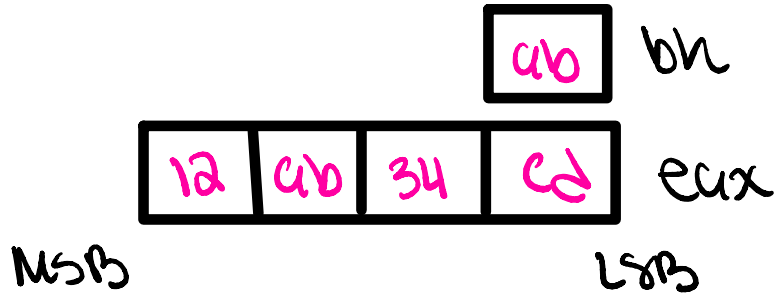


Little Endian Example

```
mov [addr], 0x12ab34cd
```

```
mov bh, [addr+2]
```

```
mov ecx, [addr]
```

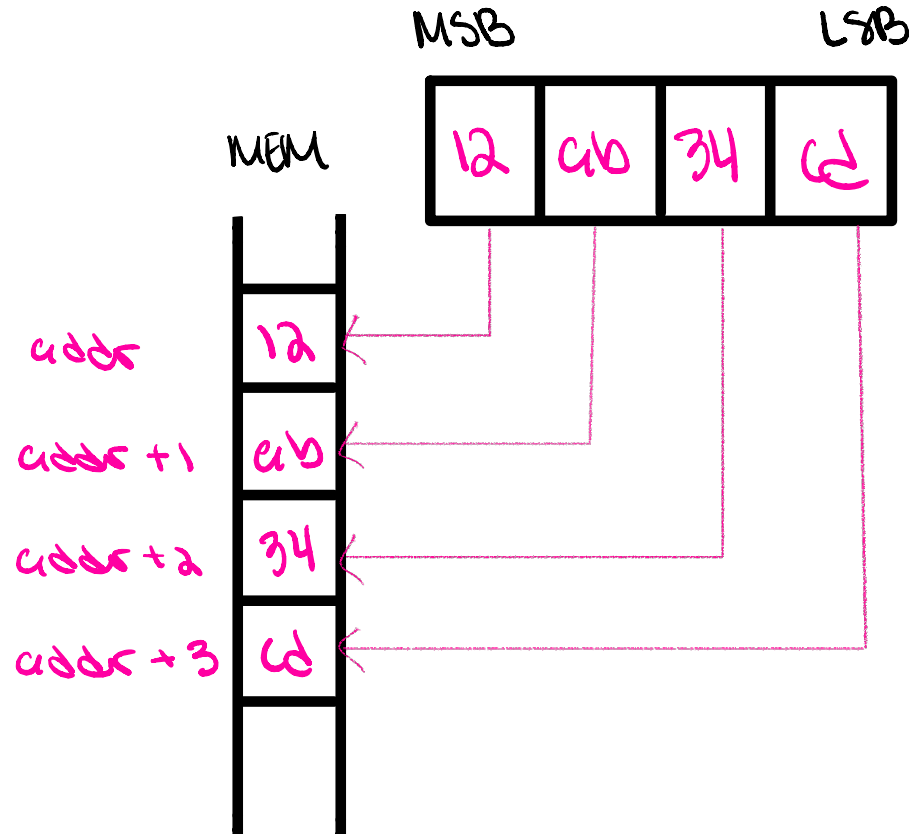
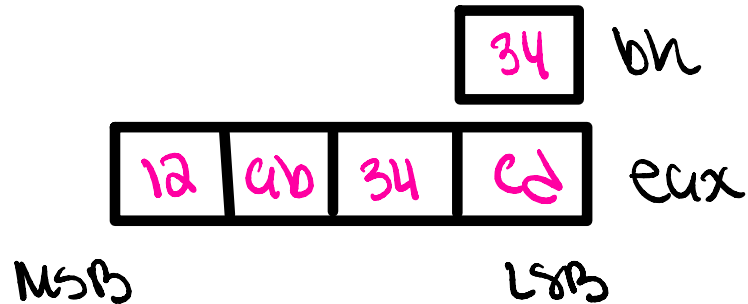


Big Endian Example

```
mov [addr], 0x12ab34cd
```

```
mov bh, [addr+2]
```

```
mov ecx [addr]
```



Intel Directives (review from last week)

Please memorize!

	size	register examples	directive
byte	8 bits	ah, al, bh, bl, ...	db
word	16 bits (two bytes)	ax, bx, cx, dx, ...	dw
dword	32 bits (four bytes)	eax, ebx, ecx, edx,	dd
qword	64 bits (eight bytes)	rax, rbx, rcx, rdx,	dq

Intel Directives (review from last week)

Directives provide instructions to the assembler and are not translated into machine language

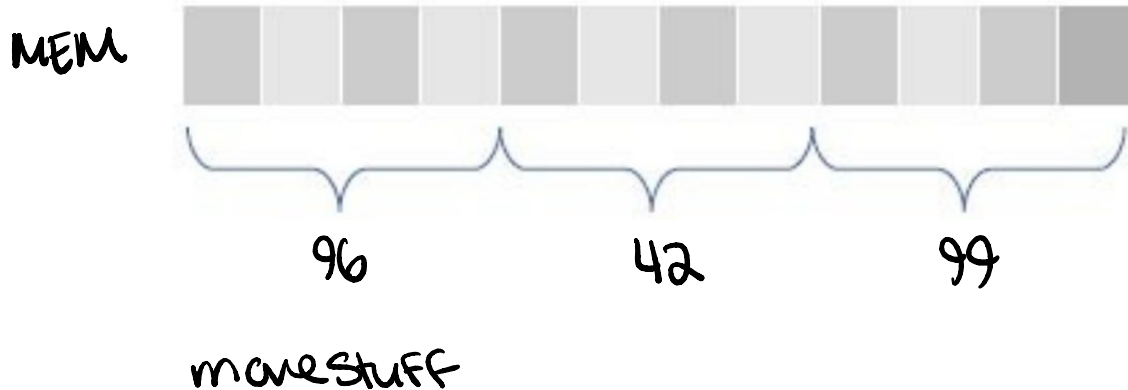
section	Subsequent assembly is applied to one of the executable's sections (usually text, data, or bss)
db, dw, dd, dq	Insert initialized literal data (of bytes, word, dword, qword)
resb, resw, resd, resq	Reserve space for uninitialized data
global	Mark a symbol as global, i.e. visible outside of this linker unit
equ	Define compile-time symbol to a particular value
;	comment

Directive Example

morestuff: dd 96, 42, 99

↳ dword (size 32 bits or 4 bytes)

↳ each element requires 4 bytes of mem
(where each addr is 1 byte)



Call Stack

The stack is section of memory that we can push and pop from for managing local variables/parameters for functions. This is to ensure functions return to the caller.

Instruction	What it does (conceptually)
JMP addr	MOV EIP, addr
PUSH reg	SUB ESP, 4 MOV [ESP], reg
POP reg	MOV reg, [ESP] ADD ESP, 4
CALL addr	PUSH EIP JMP addr
RET	POP EIP
RET n	POP EIP ADD ESP, n

Stack

- Prologue:

- `push ebp` ; save the caller's base pointer
- `mov ebp, esp` ; set our base pointer
- `sub esp, X` ; allocate X bytes for local var

- Epilogue:

- `mov esp, ebp` ; de-allocate local var
- `pop ebp` ; restore caller's base pointer
- `ret N` ; return, and pop N bytes of params

Syscalls

- Interface to operating system from application
- Provides services such as:
 - File access (open, read, write, close, seek, etc)
 - Network access (socket, accept, listen, bind, etc)
 - Directory access (mkdir, rmdir, rename, etc)
 - Process management (exec, kill, getpid, etc)
 - Memory allocation (brk, mmap, etc)
 - Etc
- In x86, It is invoked by “**int 80h**” (in hex)

	eax	Name	Source	ebx	ecx	edx
*	1	sys_exit	kernel/exit.c	int	-	-
	2	sys_fork	arch/i386/kernel/ process.c	struct pt_regs	-	-
*	3	sys_read	fs/read_write.c	unsigned int	char *	size_t
*	4	sys_write	fs/read_write.c	unsigned int	const char *	size_t

The number
of the system
call goes
in eax

The file handle
goes in ebx

The address
of the data
goes in ecx

The number
of bytes to
write goes in
edx

You get a file handle:

- by opening a file
- by creating a network socket
- in addition:
 - 0 = stdin
 - 1 = stdout
 - 2 = stderr